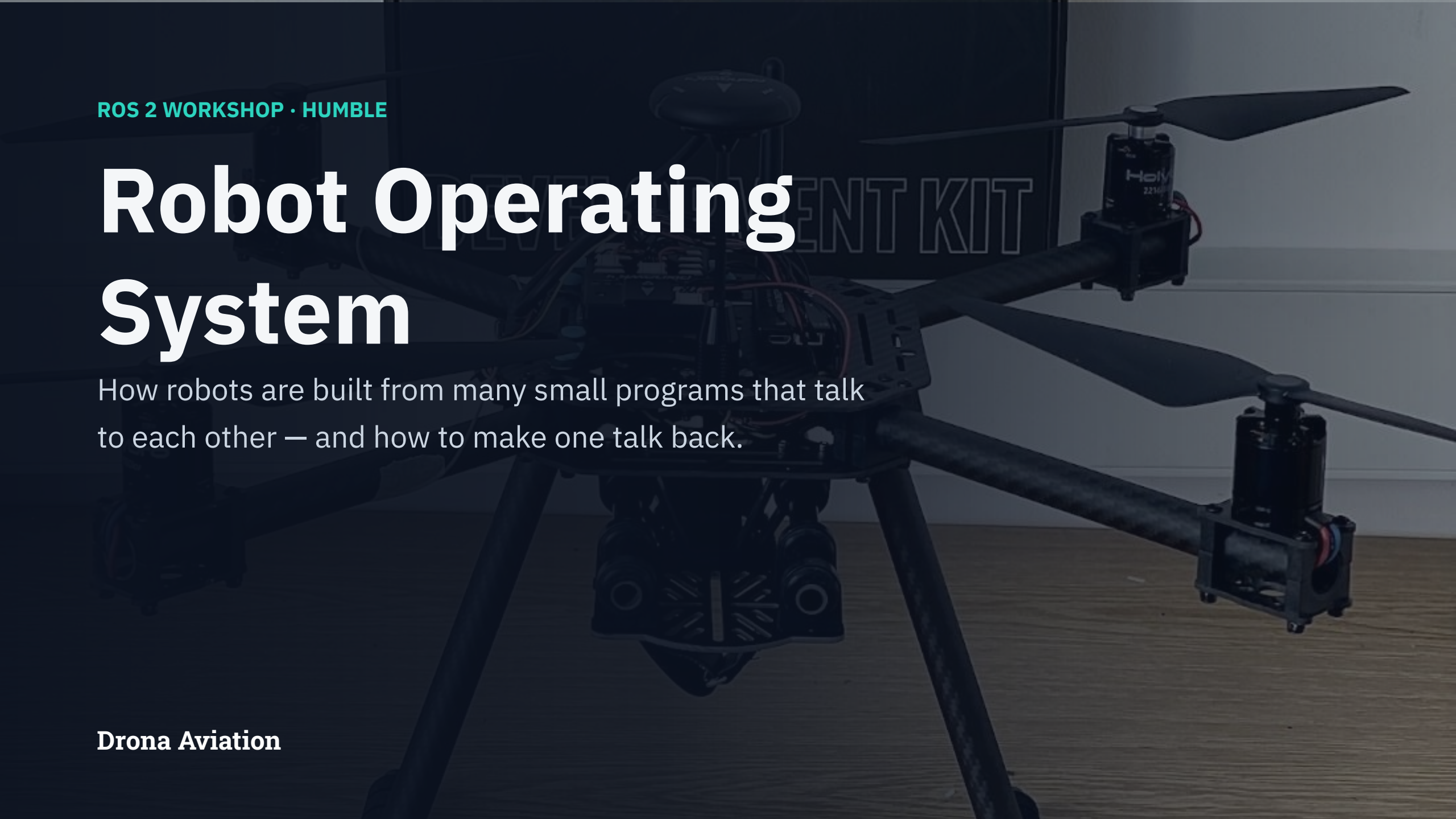
ROS 2 WORKSHOP · HUMBLE

# Robot Operating System

How robots are built from many small programs that talk to each other — and how to make one talk back.



**Drona Aviation**

# Not an operating system. A way to build a robot out of small programs.

## What it is

- A set of **libraries** your code imports (rclpy, rclcpp)
- **Command-line tools** to inspect a running robot
- **Standard message types** every robot shares (Twist, Image, LaserScan)
- Thousands of **ready-made programs**: drivers, simulators, planners

## Why the name

- It does what an OS does for apps: **process management, messaging, hardware abstraction, packaging**
- But it **runs on** Ubuntu, not instead of it
- Started at Stanford and Willow Garage in 2007; open source (Apache 2.0 for ROS 2)

> **One sentence to remember:** a robot running ROS is not one program but many small ones that pass messages to each other.

# A drone is a camera, an IMU, a radio, four motors and a battery. Who writes the code?

### One big program

- One bug stops everything
- Two people cannot work on it at once
- Camera code has to know how a motor works
- Nothing can be reused on the next robot

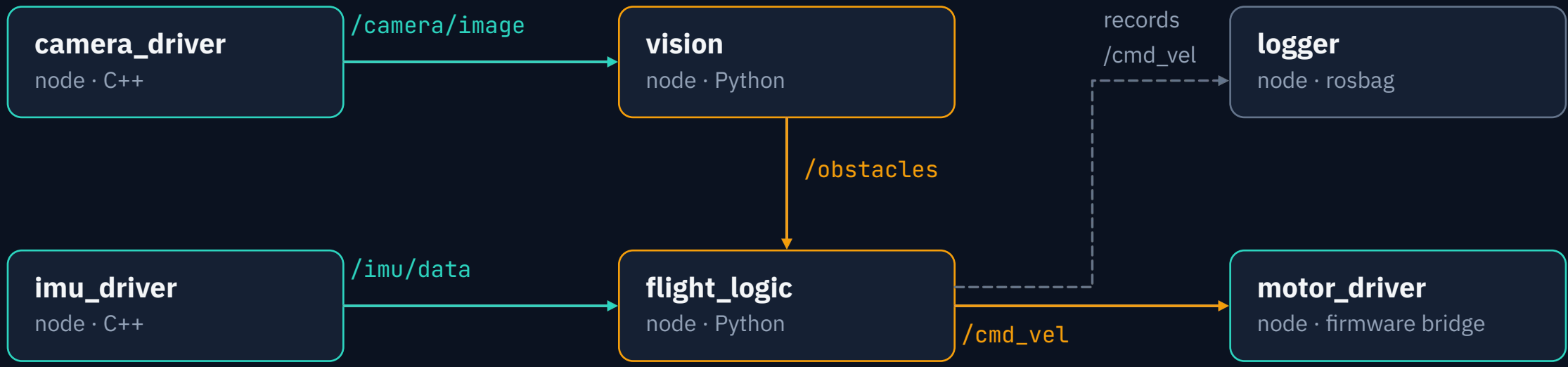
### Many small programs on ROS

- A camera node can fail on its own — *if* the rest of the system and its failsafes are designed for it
- Vendor writes the motor driver in C++, you write behaviour in Python
- Swap the camera: only one program changes
- Same code drives a turtle, a simulated drone, a real drone



TurtleBot 4: a lidar, a depth camera, a mobile base and a Raspberry Pi — each a separate ROS 2 program from a different vendor, working as one robot.

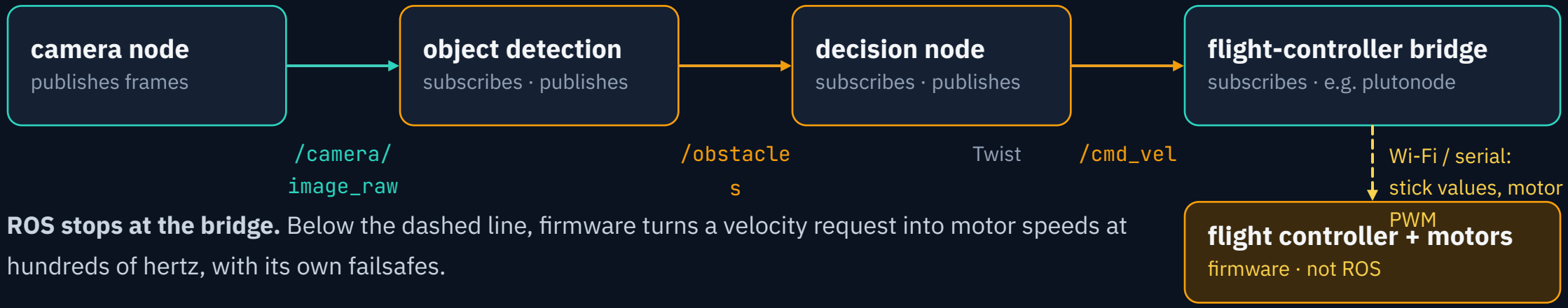
# Nodes do one job each. Topics carry messages between them. Nobody needs to know who else exists.




Each arrow is a **topic**: a named channel with a fixed message type. A publisher sends without knowing who listens; a subscriber listens without knowing who sends. The logger joins later and nothing else changes.

3B · ONE COMPLETE CHAIN ON A DRONE

# From a camera frame to spinning motors — and where ROS stops




**ROS stops at the bridge.** Below the dashed line, firmware turns a velocity request into motor speeds at hundreds of hertz, with its own failsafes.

| Node | Subscribes to | Publishes | Rate |
|---|---|---|---|
| camera node | — | /camera/image_raw (sensor_msgs/Image) | 30 Hz |
| object detection | /camera/image_raw | /obstacles (custom message) | 10–30 Hz |
| decision node | /obstacles | /cmd_vel (geometry_msgs/Twist) | 10 Hz |
| flight-controller bridge | /cmd_vel | — (talks to firmware, not a topic) | firmware: 100s of Hz |

# Same ideas, rebuilt for real robots outside the lab.

ROS 1 · 2007 – 2025

## Built for research

- One central **master** (roscore) that every node registers with
- Its own network protocol (TCPROS); one robot, one trusted network
- Python 2 era; Linux only in practice
- Last release, **Noetic**, reached end of life in May 2025

ROS 2 · 2017 – TODAY

## Built for products

- **No master.** Nodes find each other directly over DDS, an industry middleware standard
- Quality-of-service settings, security, multi-robot, lossy Wi-Fi
- Python 3, C++17; Linux, Windows, macOS
- **Humble** (2022) is the long-term-support release we use: supported to May 2027

# What changed, line by line

| Aspect | ROS 1 (Noetic, EOL 2025) | ROS 2 (Humble, supported to 2027) |
|---|---|---|
| Architecture | Central master (roscore) | No master; fully distributed |
| Communication | Custom TCPROS / UDPROS | DDS, an industry standard; vendor swappable |
| Discovery | Through the master's address | Peer-to-peer; ROS_DOMAIN_ID, ROS_LOCALHOST_ONLY |
| Quality of service | Only a queue size | Per topic: reliability, durability, deadline… |
| Real-time | Not designed for it | Possible with an RT kernel; not the default |
| Security | None built in | SROS2: authentication and encryption |
| Platforms | Ubuntu Linux in practice | Ubuntu 22.04 Tier 1; Windows, macOS, micro-ROS |

# A node is one running program with one job.

**`turtlesim_node`**
draws the turtle, moves it, reports its pose

**`turtle_teleop_key`**
reads your arrow keys, sends velocity

**`camera_driver`**
grabs frames from a USB or Wi-Fi camera

**`plutonode`**
talks to the Pluto drone's firmware over Wi-Fi

- Written in Python (**rclpy**) or C++ (**rclcpp**); a robot mixes both freely
- Many nodes run at once, on one computer or several
- A node can publish, subscribe, offer services and call them — all at the same time

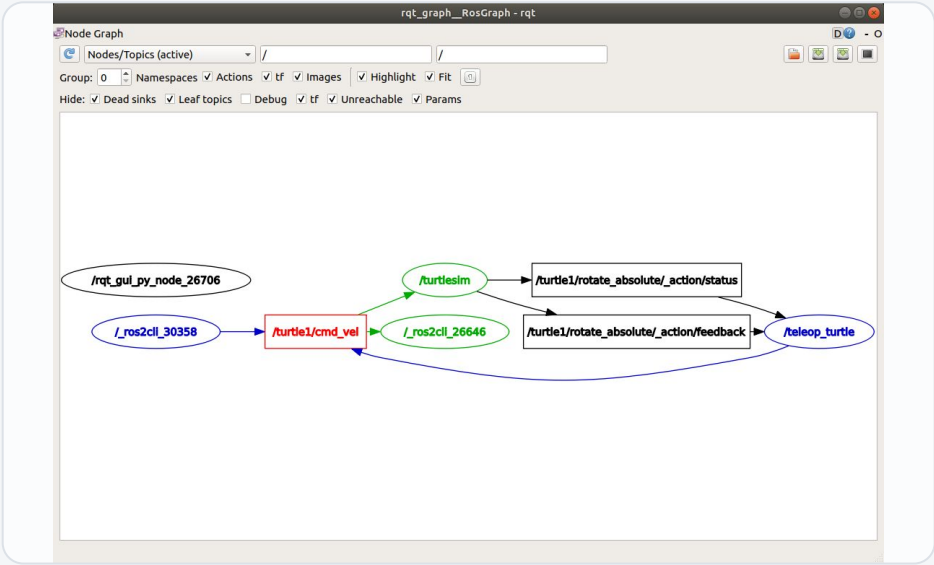
```
$ ros2 run turtlesim turtlesim_node
[INFO] [turtlesim]: Starting turtlesim with node name /turtlesim
$ ros2 node list
/teleop_turtle
/turtlesim
$ ros2 node info /turtlesim
Subscribers: /turtle1/cmd_vel
Publishers: /turtle1/pose, /turtle1/color_sensor
Service Servers: /reset, /spawn, /clear …
Action Servers: /turtle1/rotate_absolute
```

ros2 run <package> <executable> starts a node. node info is the map of everything it does.

# A topic is a named channel for a stream of messages.

- Think of a **group chat with a name**: anyone can post, anyone can read
- Every message on it has the **same fixed shape** (its message type)
- For things that **keep happening**: positions at 62 Hz, camera frames at 30 Hz, velocity commands at 10 Hz
- A typo in the name makes a new, empty topic: /turtle1/cmd_vel and /turtle1/cmdvel never meet

```
$ ros2 topic list -t
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
$ ros2 topic hz /turtle1/pose
average rate: 62.5
```




rqt_graph: ovals are nodes, boxes are topics, arrows show who publishes and who subscribes. This is the live picture of the running system.

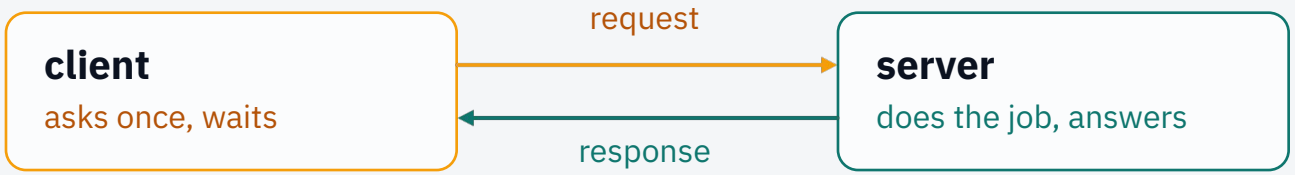
# One publisher, any number of subscribers, one message type they all agree on.




```
$ ros2 interface show geometry_msgs/msg/Twist
Vector3 linear   # x y z  m/s
Vector3 angular  # x y z  rad/s
```

The **message** is a form with named boxes. A publisher fills it in and calls publish(); ROS calls each subscriber's **callback** with a copy. Neither side knows the other's name — only the topic and the type.

# A service is one request and one reply. A phone call, not a broadcast.




```
$ ros2 interface show turtlesim/srv/Spawn
float32 x, float32 y, float32 theta, string name  # request
---
string name  # response
$ ros2 service call /spawn turtlesim/srv/Spawn "{x: 2.0, y: 2.0,
theta: 0.0, name: 'turtle2'}"
response: turtlesim.srv.Spawn_Response(name='turtle2')
```

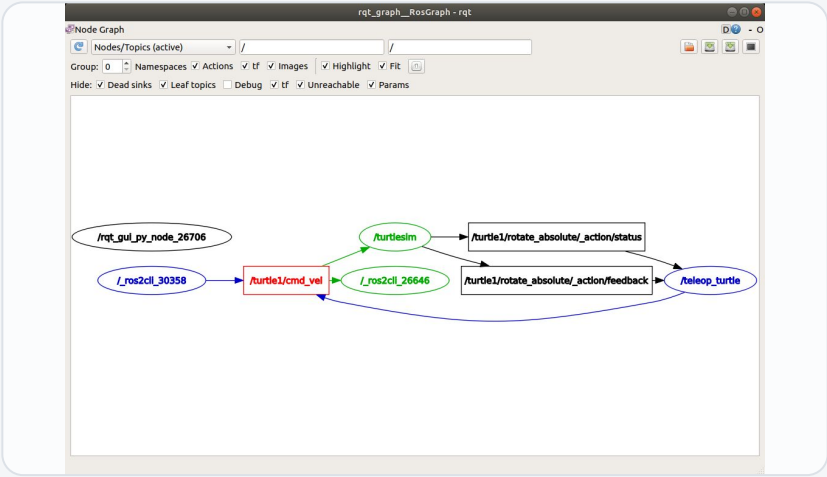



- Request fields above the ---, response fields below
- Use for things that happen **once** and need an answer: spawn, reset, take a photo, save the map
- Turtlesim services: /spawn, /reset, /clear, /kill

# An action is a long task: send a goal, watch the feedback, get a result — or cancel.




```
$ ros2 action send_goal /turtle1/rotate_absolute
turtlesim/action/RotateAbsolute "{theta: 1.57}" --feedback
Feedback: remaining: 1.20 … 0.60 … 0.08
Result: delta: -1.57
Goal finished with status: SUCCEEDED
```




- Three parts in the .action file: goal, result, feedback
- Built from services and topics underneath — rqt_graph shows the feedback and status topics
- Real uses: "navigate to the kitchen", "take off to 2 m", "pick up the box"

# Four ways nodes communicate. Pick by the shape of the conversation.

| Kind | Pattern | Reply | Example |
|---|---|---|---|
| Topic | Stream, many-to-many | No | /turtle1/pose · camera frames |
| Service | One request, one reply | One | /spawn · "take a photo" |
| Action | Goal, feedback, result, cancel | Progress + final | /turtle1/rotate_absolute · "navigate to X" |
| Parameter | A node's setting, live | — | background_r · camera frame rate |

```
// parameters demo: set all three channels, then redraw
$ ros2 param set /turtlesim background_r 255
$ ros2 param set /turtlesim background_g 0
$ ros2 param set /turtlesim background_b 0
$ ros2 service call /clear std_srvs/srv/Empty
→ the background turns red
```

The three background_* parameters (0–255) decide the colour; they are read when the window repaints, and /clear forces a repaint. Set all three, or you get whatever the other two already were.
An interface file (.msg .srv .action) is a list of named, typed fields both sides share.

# Six tools you will use on every robot

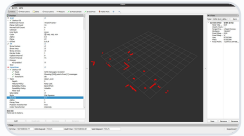
## ros2 CLI

Inspect and poke the live graph from any terminal.

```
ros2 node list
ros2 topic echo /turtle1/pose
ros2 service call /reset …
```

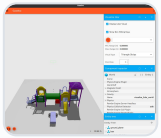
## RViz2

3D view of what the robot believes: frames, lidar points, camera images, paths.



## Gazebo

Physics simulator: gravity, collisions, sensors. Crashes are free. A bridge links it to ROS.



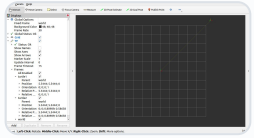
## rqt

GUI plugins: rqt_graph draws nodes and topics; others plot topics, show images, call services.

```
rqt_graph
```

## TF2

Tracks coordinate frames over time: camera relative to base, base relative to map.



## rosbag

Record topics to disk, play them back later. Debug a crash without the robot.

```
ros2 bag record /turtle1/pose
ros2 bag play <folder>
```

# Code lives in a package, packages live in a workspace, colcon builds it, launch starts it.

```
~/ros2_ws/              workspace
├── src/
│   └── my_pkg/         package
│       ├── package.xml
│       ├── setup.py
│       ├── my_pkg/
│       │   ├── talker.py
│       │   └── listener.py
│       └── launch/
│           └── demo.launch.py
├── build/              made by colcon
├── install/            you source this
└── log/
```

1. **Create** ros2 pkg create --build-type ament_python my_pkg --dependencies rclpy
2. **Declare** package.xml lists dependencies; setup.py entry_points turns talker.py into ros2 run my_pkg talker
3. **Build** cd ~/ros2_ws && colcon build --symlink-install
4. **Source** source install/setup.bash in every new terminal
5. **Launch** ros2 launch my_pkg demo.launch.py starts several nodes from one file

> demo.launch.py is a Python function returning a list of Node(package=..., executable=...). Real robots start dozens of nodes; nobody opens dozens of terminals.

14 · WHERE ROS IS USED

# The same nodes-and-topics idea, from a toy to a factory



## Drones

PX4 and ArduPilot bridge to ROS 2; offboard control, mapping, inspection. Our Pluto: plutonode + a 10-line publisher.



## Autonomous mobile robots

Nav2 for navigation, SLAM Toolbox for mapping: warehouses, hospitals, delivery.



## Robotic arms & industry

MoveIt 2 for motion planning; ros2_control for drivers; ROS-Industrial for Universal Robots, ABB, KUKA.



## Vision, fleets & research

OpenCV via cv_bridge, multi-robot with domain IDs, and the default platform in robotics papers and labs.

Common thread: a sensor publishes, a planner subscribes and publishes, a driver subscribes. Change the robot, keep the pattern.

# Drive a turtle, then look inside it

**0 · FIRST, IN EVERY TERMINAL**

```
source ~/ROS2_Tutorial/env.sh
```

**1 · T1**

```
ros2 run turtlesim turtlesim_node
```

Expected: a blue window with a turtle in the middle

**2 · T2**

```
ros2 run turtlesim turtle_teleop_key
```

Click inside T2, then arrow keys. Expected: the turtle moves and draws a line

**3 · T3**

```
ros2 node list
```

Expected: /teleop_turtle and /turtlesim

**4 · T3**

```
ros2 topic list -t
```

Expected: /turtle1/cmd_vel [geometry_msgs/msg/Twist] and four more

**5 · T3**

```
ros2 topic info /turtle1/cmd_vel
```

Expected: Publisher count: 1 (keyboard), Subscription count: 1 (turtlesim)



**Before the session:** bash ~/ROS2_Tutorial/check_setup.sh must end with *All checks passed*.

**Keys do nothing?** T2 must be the active window. The turtle window ignores the keyboard.

# Become the keyboard

**6 · T3**

```
ros2 topic echo /turtle1/cmd_vel
```

Press ↑ once in T2. Expected: one Twist with linear x: 2.0; ← gives angular z: 2.0. Ctrl+C to stop

**7 · T3**

```
ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z: 1.8}}"
```

Expected: the turtle drives in circles with no keyboard. Ctrl+C: it stops within one second

**8 · T3**

```
rqt_graph
```

Expected: two ovals joined by an arrow labelled /turtle1/cmd_vel. No arrow? Choose "Nodes/Topics (active)" and refresh

**9 · T3**

```
ros2 service call /reset std_srvs/srv/Empty
```

Expected: the turtle is back in the centre and the trail is gone

**Try after step 7:** bigger z = tighter circle · negative z = clockwise · x: 0.0 = spin on the spot · --once instead of --rate 1 = moves for one second and stops

**Stuck?** Is the other node running (ros2 node list)? Is the topic name exact (ros2 topic list)? Did this terminal run the setup line?

# A robot is many small programs that pass messages. Everything else is detail.

## Six words to keep

| | |
|---|---|
| Node | One running program, one job |
| Topic | A named stream; publishers and subscribers never meet |
| Message | The fixed shape every post on a topic must have |
| Service | One request, one reply |
| Action | A goal with feedback, a result, and cancel |
| Package | Where code lives; colcon builds it, launch starts it |

ROS 2 = no master, DDS underneath, QoS per topic, real-time *possible* with the right kernel and setup. Humble on Ubuntu 22.04.

```
sudo apt update && sudo apt install -y git
```

```
git clone https://github.com/DronaAviation/ROS2_Tutorial26.git
~/ROS2_Tutorial
```